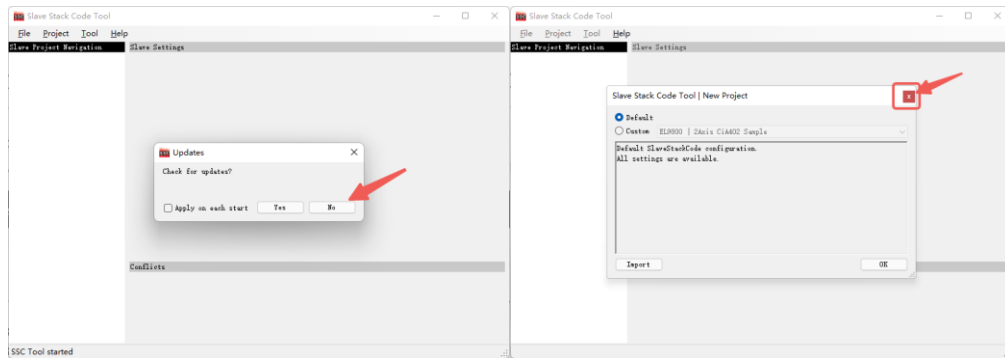
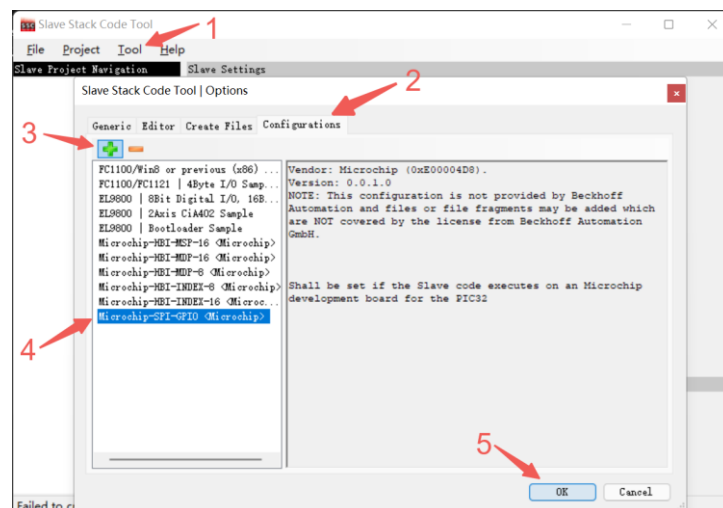


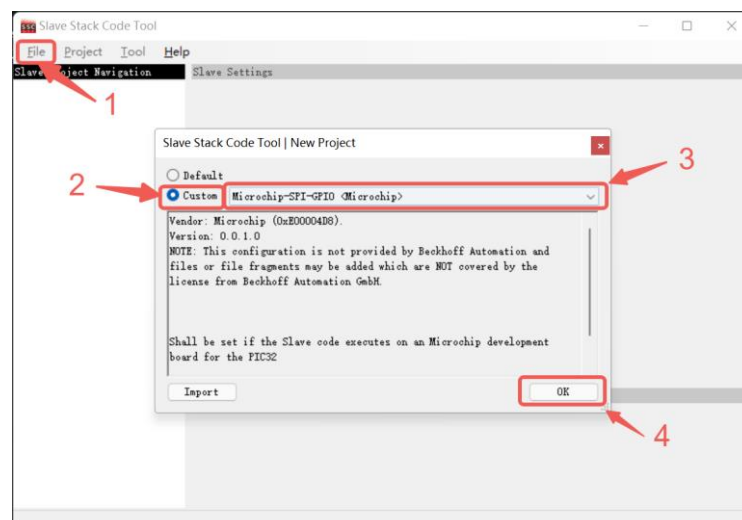
- 1 打开 SSC Generate LAN9252_EtherCAT Protocol Stack\Tool Software\SSC_V5i13 文件夹, 安装 EtherCAT Slave Stack Code Tool.exe, 打开安装好的 SSC Tool 软件, 按步骤操作;



- 2 导入 Microchip 配置文件
工具栏 Tool -> Options -> Configuration -> "+" 选择 《***\tools\SSC Generate LAN9252_EtherCAT Protocol Stack\Tool Software\LAN9252-PIC32-SDK-1.1》路径文件夹下的 Microchip_LAN9252_SSC_Config.xml 文件, 选中"Microchip-SPI-GPIO <Microchip>"选项后点击 OK

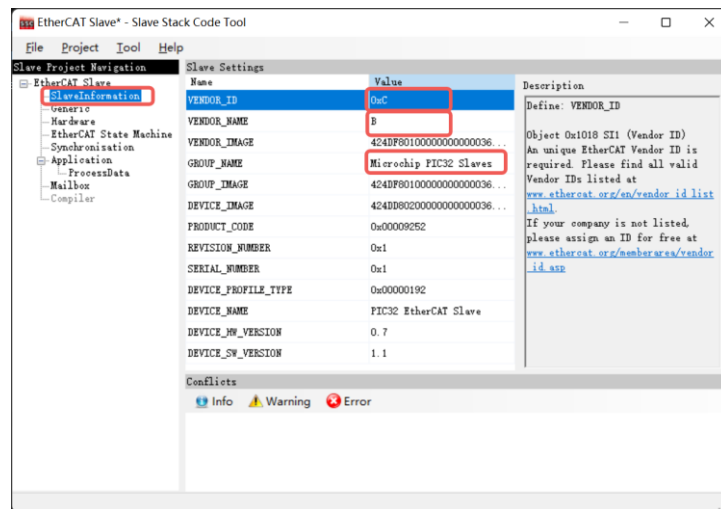


- 3 创建基于 LAN9252 的 SPI 项目
工具栏 File->New->Custom->Microchip-SPI-GPIO<Microchip>, 点击 OK 后, 弹出对话框, 点击 YES

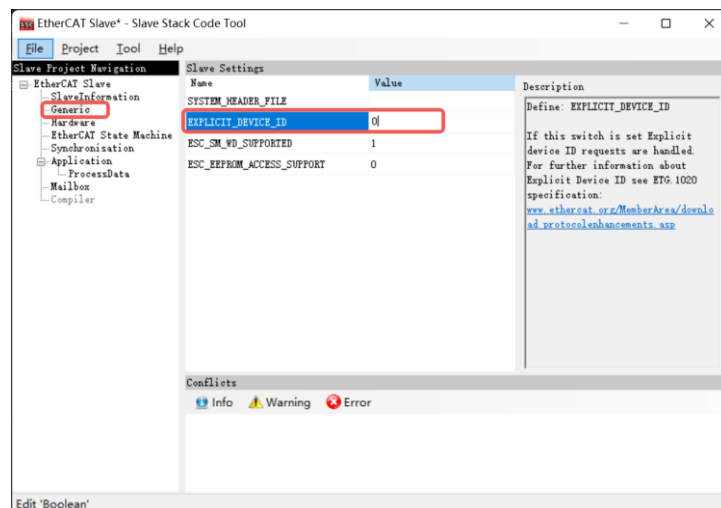


4 根据应用需求修改项目信息

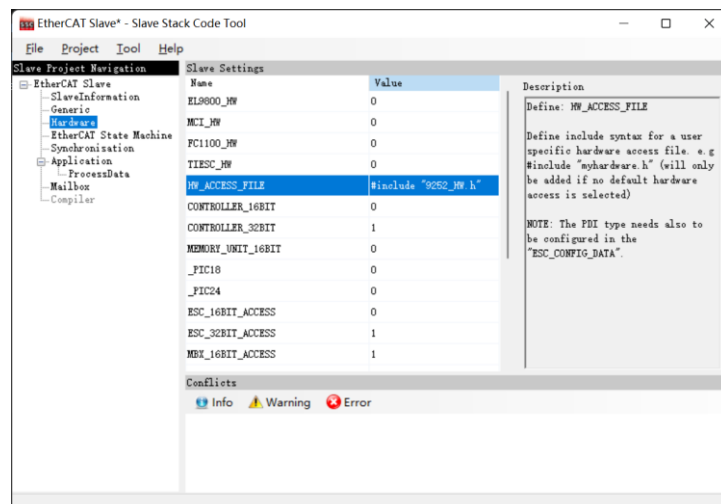
- 4.1 点击 SlaveInformation, 按需修改厂商 ID (如 0x87654321)、名称 (如 CompanyX) 及项目名称 (如 Demo), 确保不同设备的 PRODUCT_CODE 唯一;



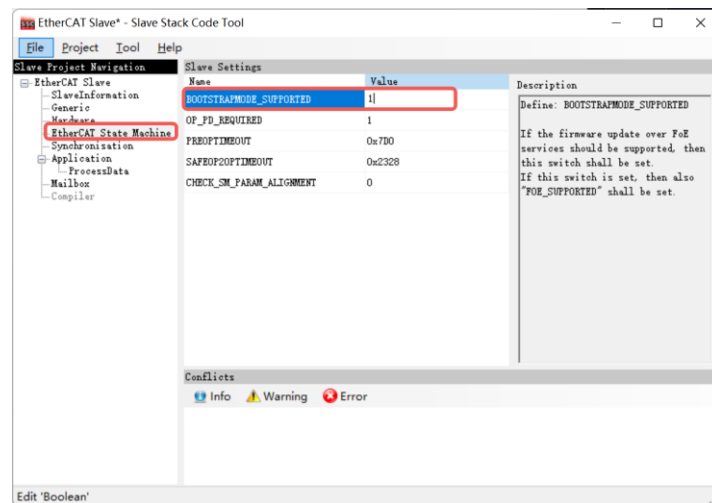
- 4.2 点击 Generic, 可将 EXPLICIT_DEVICE_ID 修改为 0, 使得主控动态分配设备 ID;



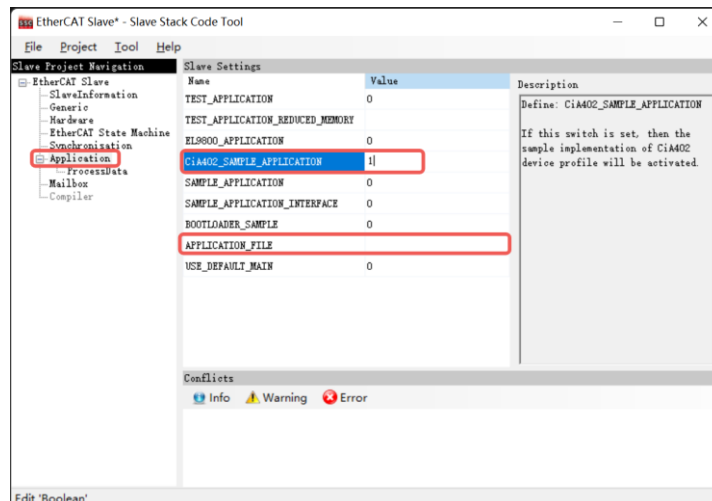
- 4.3 点击 Hardware, 修改 HW_ACCESS_FILE 为#include "STM32_LAN9252_HW", 因为我们会有自己的驱动文件;



- 4.4 点击 EtherCAT State Machine，如果需要支持 FOE，将 BOOTSTRAPMODE_SUPPORTED 置 1；



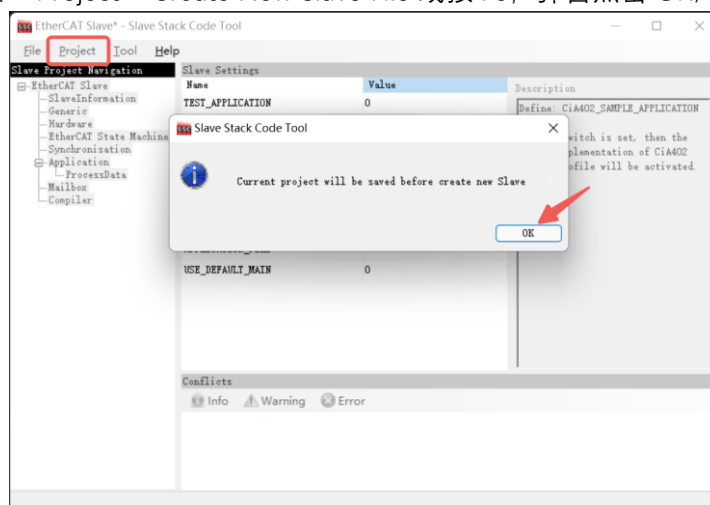
- 4.5 点击 Application，删除 APPLICATION_FILE 内容，如果需要使用默认的 CiA402 历程，CiA402_SAMPLE_APPLICATION 置 1；



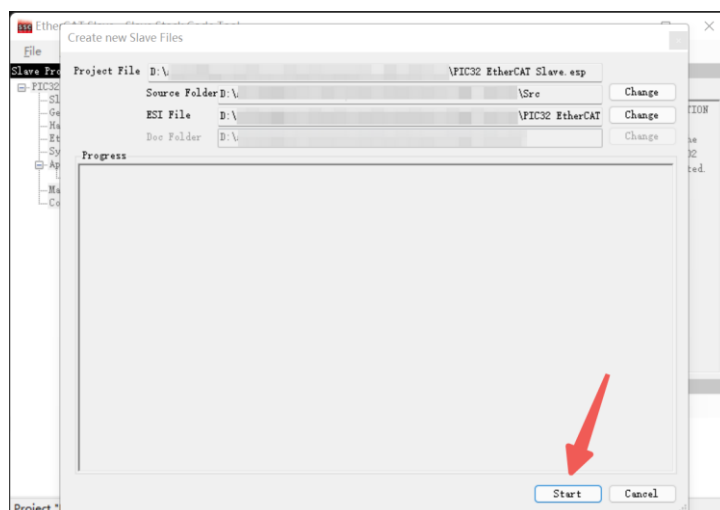
- 4.6 其他配置保持默认，有其他特殊要求则自行更改。

5 生成从站代码

5.1 工具栏->Project->Create New Slave File 或按 F5，弹窗点击 OK；



5.2 创建文件夹并保存项目，点击 Start 开始生成，等待完成后，弹出对话框，点击 OK；

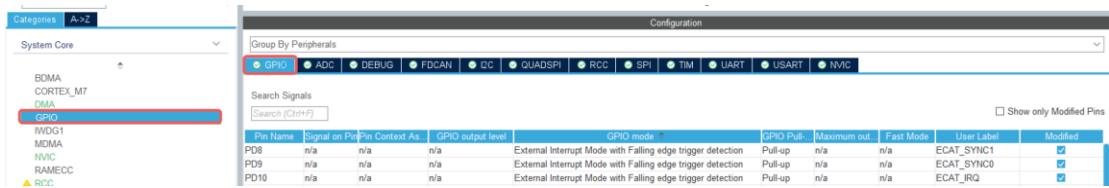


5.3 打开《***\tools\SSC Generate LAN9252_EtherCAT Protocol Stack\BSP Driver》路径，根据硬件接口选择对应的 QSPI 或 SPI 文件夹下 Port 文件夹，拷贝到代码生成的 Src 文件夹相同路径下；



6 嵌入式软件 STM32CubeMX 配置（其他硬件平台可参考此步骤进行相应配置）

6.1 配置 LAN9252 使用到的三组外部中断 GPIO，下降沿触发；



修改驱动代码的“STM32_LAN9252_HW.h”文件，使之与 User Label 名称进行对应关联；

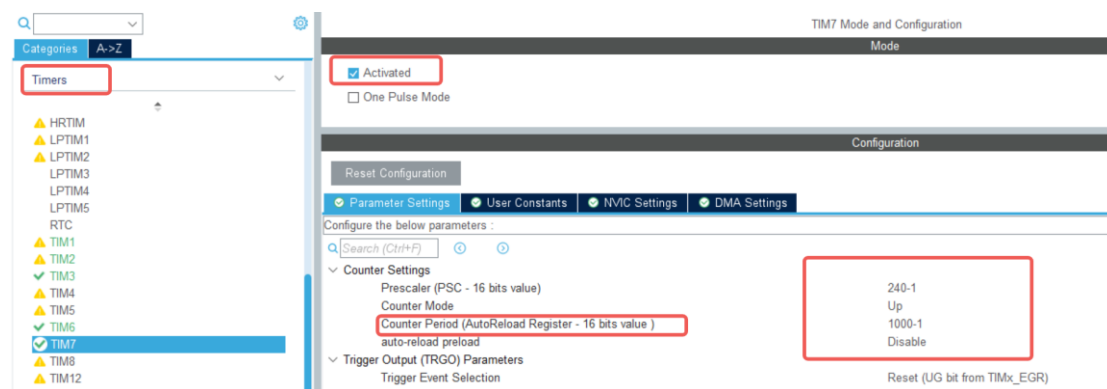
```
/*-----  
----- SYNC0中断  
-----  
-----*/  
#define LAN9252_SYNC0_PIN ECAT_SYNC0_Pin  
#define LAN9252_SYNC0_IRQ ECAT_SYNC0_EXTI_IRQn  
  
#define INIT_SYNC0_INT()  
#define DISABLE_SYNC0_INT() HAL_NVIC_DisableIRQ(LAN9252_SYNC0_IRQ)  
#define ENABLE_SYNC0_INT() HAL_NVIC_EnableIRQ(LAN9252_SYNC0_IRQ)  
  
/*-----  
----- SYNC1中断  
-----  
-----*/  
#define LAN9252_SYNC1_PIN ECAT_SYNC1_Pin  
#define LAN9252_SYNC1_IRQ ECAT_SYNC1_EXTI_IRQn  
  
#define INIT_SYNC1_INT()  
#define DISABLE_SYNC1_INT() HAL_NVIC_DisableIRQ(LAN9252_SYNC1_IRQ)  
#define ENABLE_SYNC1_INT() HAL_NVIC_EnableIRQ(LAN9252_SYNC1_IRQ)
```

```
/*-----  
----- ESC中断  
-----  
-----*/  
#define LAN9252_IRQ_PIN ECAT_IRQ_Pin  
#define LAN9252_IRQ_IRQ ECAT_IRQ_EXTI_IRQn  
  
#define INIT_ESC_INT()  
#define DISABLE_ESC_INT() \\\n{\n    HAL_NVIC_DisableIRQ(LAN9252_IRQ_IRQ); \\\n    DISABLE_SYNC0_INT(); \\\n    DISABLE_ECAT_TIMER_INT(); \\\n}\n#define ENABLE_ESC_INT() \\\n{\n    HAL_NVIC_EnableIRQ(LAN9252_IRQ_IRQ); \\\n    ENABLE_SYNC0_INT(); \\\n    ENABLE_ECAT_TIMER_INT(); \\\n}
```

✧ 若使用 YuanHub 开源项目修改《***\\embedded_project\\Y01\\Board》路径下的 board.h 文件，使之与 User Label 名称进行对应关联；

```
h board.h 9+ x  
embedded_project > Y01 > Board > h board.h > ...  
121  
122 #define ECAT_IRQ_EXTI_LINE (ECAT_IRQ_Pin)  
123 #define ECAT_IRQ_EXTI_IRQn (ECAT_IRQ_EXTI_IRQn)  
124 #define ECAT_SYNC0_EXTI_LINE (ECAT_SYNC0_Pin)  
125 #define ECAT_SYNC0_EXTI_IRQn (ECAT_SYNC0_EXTI_IRQn)  
126 #define ECAT_SYNC1_EXTI_LINE (ECAT_SYNC1_Pin)  
127 #define ECAT_SYNC1_EXTI_IRQn (ECAT_SYNC1_EXTI_IRQn)  
128 #define ECAT_EXTI_IRQ_TASK HAL_GPIO_EXTI_Callback
```

6.2 选择一个定时器，配置 EtherCAT 的 1ms 中断任务，如果配置成其他的分频系数也可以，但是必须是 1ms 中断，记住此时的 ARR 值（1000-1）；



修改驱动代码的“STM32_LAN9252_HW.h”文件，使之对应；

```
/*-----
----- 定时器中断
-----*/

#define LAN9252_TIM_HANDLE htim7
#define ECAT_TIMER_INC_P_MS 1000

#define HW_GetTimer() __HAL_TIM_GET_COUNTER(&LAN9252_TIM_HANDLE)
#define HW_ClearTimer() __HAL_TIM_SET_COUNTER(&LAN9252_TIM_HANDLE, 0)

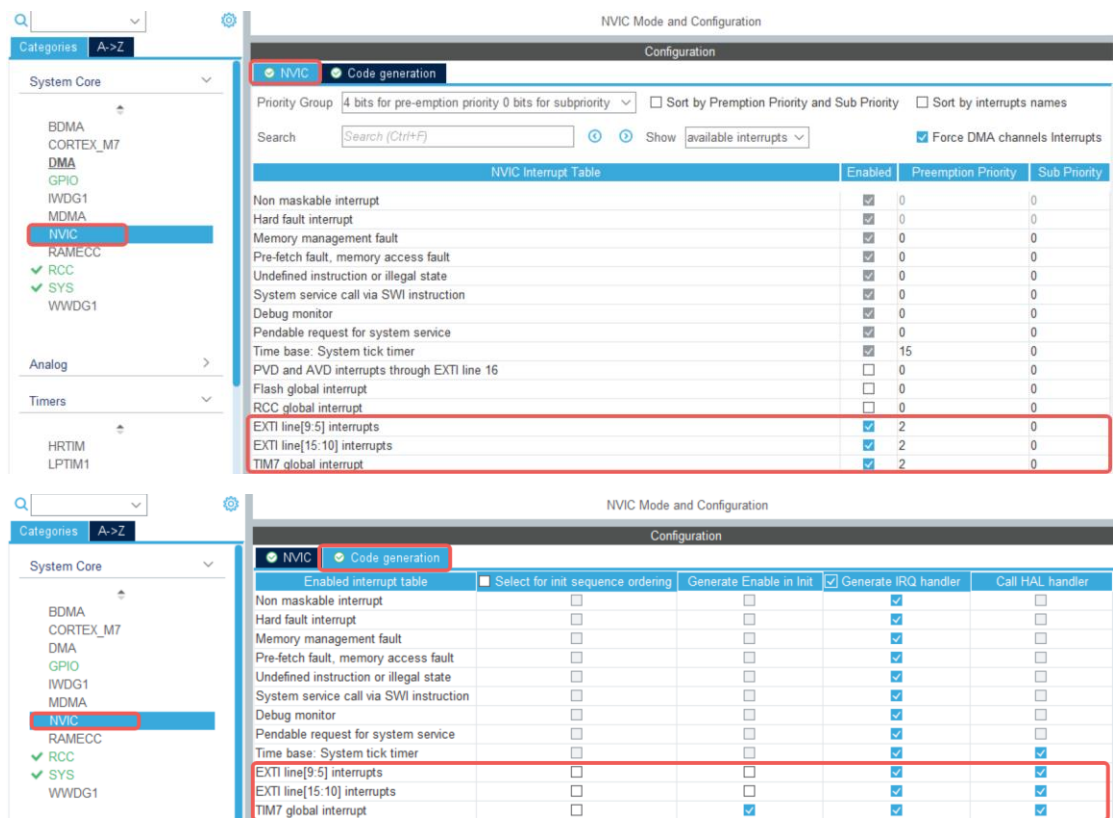
#define INIT_ECAT_TIMER() __HAL_TIM_SET_COUNTER(&LAN9252_TIM_HANDLE, 0)
#define DISABLE_ECAT_TIMER_INT() __HAL_TIM_DISABLE_IT(&LAN9252_TIM_HANDLE, TIM_IT_UPDATE);
#define ENABLE_ECAT_TIMER_INT() __HAL_TIM_ENABLE_IT(&LAN9252_TIM_HANDLE, TIM_IT_UPDATE);
#define STOP_ECAT_TIMER() HAL_TIM_Base_Stop_IT(&LAN9252_TIM_HANDLE)
#define START_ECAT_TIMER() HAL_TIM_Base_Start_IT(&LAN9252_TIM_HANDLE)
```

✧ 若使用 YuanHub 开源项目修改《***\embedded_project\Y01\Board》路径下的 board.h 文件，使之对应；

```
h board.h 9+ x
embedded_project > Y01 > Board > h board.h ...
77
78 #define BRAKE_PWM_TIM_HANDLE (htim2) //PWM抱闸IO输出定时器
79 #define ENCODER_ABZ_TIM_HANDLE (htim3) //ABZ增量式编码器定时器
80 #define ENCODER_ABZ_TIM_Z_CHANNEL (TIM_CHANNEL_3) //ABZ增量式编码器定时器z相捕获通道
81 #define CANOPEN_TIM_HANDLE (htim6) //CANopen定时器
82 #define ECAT_LAN9252_TIM_HANDLE (htim7) //EtherCAT定时器
83 #define NRT_TASK_TIM_HANDLE (htim13) //非实时(1ms)任务定时器
84 #define NRT_CAN_ECAT_IRQ_TASK HAL_TIM_PeriodElapsedCallback
```

6.3 相关中断优先级配置

一共 4 个中断应配置成优先级 TIM=IRQ>SYNC0=SYNC1，若无法配置成这种，应配置成相同优先级 TIM=IRQ=SYNC0=SYNC1，并且生成代码中，取消 Generate Enable in Init 下 IRQ/SYNC0/SYNC1 对应中断线的 Enable 框，EtherCAT 协议栈会自动打开；



6.4 配置通讯接口（根据硬件二选一）

● SPI 模式

Search (Ctrl+F)

Basic Parameters

Frame Format: Motorola

Data Size: 8 Bits

First Bit: MSB First

Clock Parameters

Prescaler (for Baud Rate): 8

Baud Rate: 21.25 MBits/s

Clock Polarity (CPOL): Low

Clock Phase (CPHA): 1 Edge

Advanced Parameters

CRC Calculation: Disabled

NSSP Mode: Disabled

NSS Signal Type: Software

配置片选引脚

PA4	n/a	High	Output Pu...	Pull-up	High	n/a	SPI1_NSS	✓
PB14	n/a	n/a	External In...	Pull-up	n/a	n/a	SYNC0	✓
PB15	n/a	n/a	External In...	Pull-up	n/a	n/a	SYNC1	✓
PA0	n/a	Low	Output Pu...	No pull-up ...	High	n/a	USART1_DE	✓

PA4 Configuration :

GPIO output level: High

GPIO mode: Output Push Pull

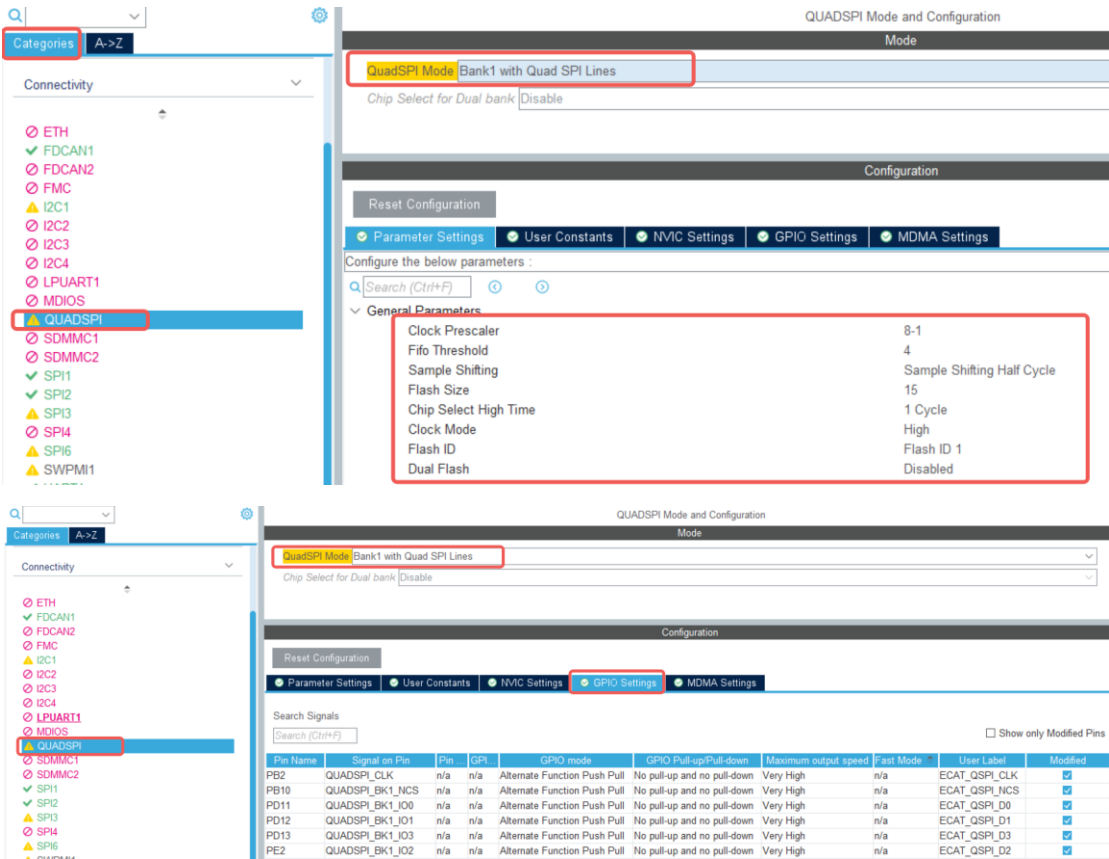
GPIO Pull-up/Pull-down: Pull-up

Maximum output speed: High

修改驱动代码的“SPIDriver.h”使之对应

```
#define LAN9252_SPI_HANDLE hspi1
#define LAN9252_SPI_NSS_PORT SPI1_NSS_GPIO_Port
#define LAN9252_SPI_NSS_PIN SPI1_NSS_Pin
```

● QSPI (QuadSPI) 模式



修改驱动代码的“SQIDriver.h”使之对应

```
/* *****宏定义***** */
#define LAN9252_SQI_HANDLE hqspi
```

✧ 若使用 YuanHub 开源项目修改《***\embedded_project\Y01\Board》路径下的 board.h 文件，使之对应；

```
h board.h 9+ x
embedded_project > Y01 > Board > h board.h > ...
86
87 #pragma region // CONNECTIVITY
88 #define CAN_FDCAN_HANDLE (hfdcan1)
89
90 #define ECAT_LAN9252_QSPI_HANDLE (hqspi)
91
```

6.5 添加协议栈任务函数到中断服务回调函数中

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == LAN9252_IRQ_PIN)
    {
        PDI_Isr();
    }
    else if (GPIO_Pin == LAN9252_SYNC0_PIN)
    {
        DISABLE_ESC_INT();
        Sync0_Isr();
        ENABLE_ESC_INT();
    }
    else if (GPIO_Pin == LAN9252_SYNC1_PIN)
    {
        Sync1_Isr();
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim == &LAN9252_TIM_HANDLE)
    {
        ECAT_CheckTimer();
    }
}
```

- ✧ 若使用 YuanHub 开源项目修改《***\embedded_project\Y01\Board》路径下的 board.c 文件，使之对应；

```
board.c 9+, M X
project > Y01 > Board > board.c > CURRENT_LOOP_IRQ_TASK > HAL_ADCEx_InjectedConvCpltCallback

void ECAT_EXTI_IRQ_TASK(uint16_t GPIO_Pin)
{
#ifdef USE_ECAT
    if (GPIO_Pin == ECAT_IRQ_EXTI_LINE)
    {
        PDI_Isr();
    }
    else if (GPIO_Pin == ECAT_SYNC0_EXTI_LINE)
    {
        DISABLE_ESC_INT();
        Sync0_Isr();
        ENABLE_ESC_INT();
    }
    else if (GPIO_Pin == ECAT_SYNC1_EXTI_LINE)
    {
        Sync1_Isr();
    }
}
#endif
}

// 1ms低频任务 典型频率 1KHZ
void NRT_CAN_ECAT_IRQ_TASK(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == NRT_TASK_TIM_HANDLE.Instance)
    {
        UnrealTimeBase1ms();
    }
#ifdef USE_ECAT
    else if (htim->Instance == ECAT_LAN9252_TIM_HANDLE.Instance)
    {
        ECAT_CheckTimer();
    }
}
#endif
}
```

6.6 根据数据字典大小适当修改生成代码的堆栈大小，建议不小于 4K；

6.7 生成代码后，打开嵌入式工程，将生成的协议栈添加到工程，编译调试。